시스템 구상도



프로젝트명	다중 AI 에이전트 기반 독자 참여형 스토리 창작 및 피드백 자동화 플랫폼
대주제	다중 에이전트 서빙기반 고객 피드백 분석 및 맞춤형 응대 자동화 플랫폼
서비스 명	STORYFORK — 원작을 나만의 이야기로.
깃허브 주소	SKN34 Final 4Team 깃허브 주소

SK 네트웍스 DATAVARE en·core DATA DRIVEN WORLD

2026년 10월 30일

SKN34 4Team

목차

목 차

Ⅰ. 시스템 아키텍처 ····· 4

Ⅱ. 시스템 구성 요소 ····· 7

제정 및 개정 이력

버전	개정 내용	작성자	승인자	개정날짜
1.0	MVP 기준 초안 작성	윤성호		2026.10.30
1.1	최신 서비스 분리 구조 반영 및 시스템 구성도·구성 요소·데이터 관리 원칙 개정	윤성호		2026.11.05

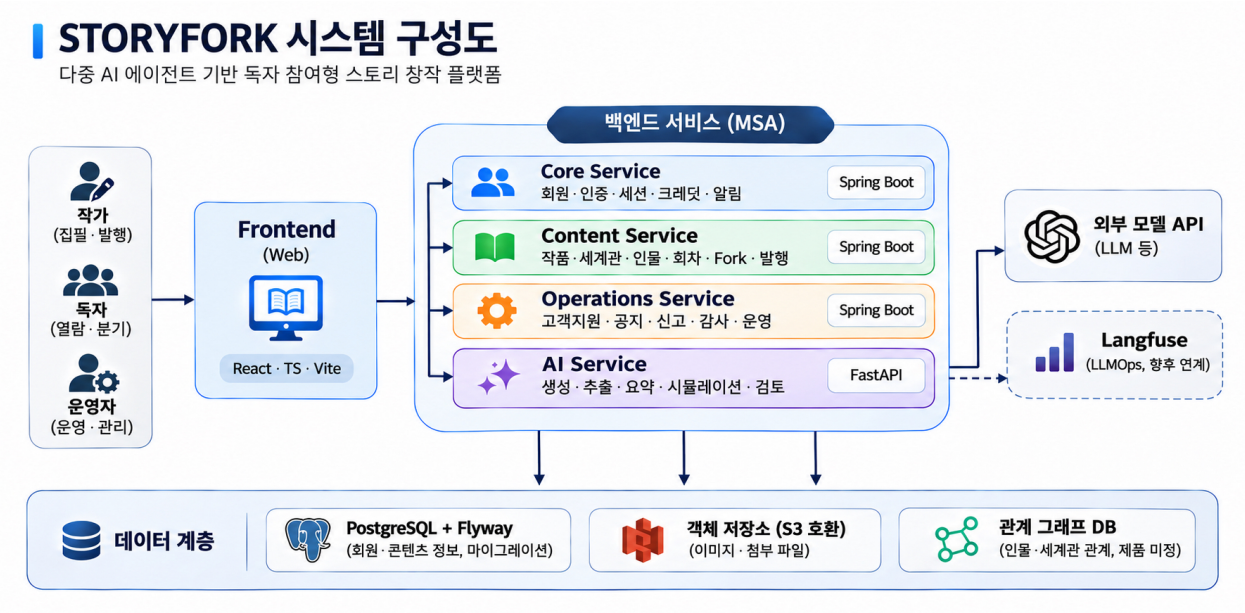
I. 시스템 아키텍처

1. 시스템 개요

STORYFORK는 다중 AI 에이전트 기반의 독자 참여형 스토리 창작 및 피드백 자동화 플랫폼이다. 작가는 세계관과 등장인물을 관리하고, 직접 집필하거나 AI의 집필 지원·다중 인물 시뮬레이션을 활용하여 이야기를 창작할 수 있다. 독자는 이용이 허용된 원작의 특정 회차에서 분기(Fork)하여 독립적인 이야기를 만들고, 작품에 대한 질문·오류·제안을 등록하여 창작 과정에 참여할 수 있다.

전체 시스템은 MSA(Microservices Architecture) 기반으로 프론트엔드와 Core·Content·Operations·AI 서비스를 분리하여 설계한다. Core는 회원·인증·크레딧·회원 알림을, Content는 작품·세계관·원고·Fork·발행 및 작품 피드백을, Operations는 고객지원·신고·운영 관리를 담당한다. AI 서비스는 허용된 문맥을 바탕으로 생성·시뮬레이션·분석 결과를 제공하며, 정본 반영과 발행 등 최종 업무 처리는 해당 업무를 소유한 서비스가 담당한다.

2. 전체 시스템 아키텍처



3. 아키텍처 설계 전략

아키텍처	적용대상	설계 목적
MSA	전체 시스템	서비스별 독립적인 개발·배포 및 유지보수
Clean Architecture + MVVM	Frontend	UI, 비즈니스 로직, 데이터 접근 계층 분리
Layered Architecture	Core·Content·Operations	Controller, Service, Repository의 역할 분리
Agentic Workflow	AI Service	LangChain·LangGraph 기반 AI 에이전트 협업 및 실행 흐름 제어

4. 주요 시스템 구성 요소

구성요소	적용기술	주요 역할
Frontend	React, TypeScript, Vite	사용자 인터페이스, 세계관 관리 및 소설 집필 화면
Core API	Spring Boot, Kotlin	사용자 인증·인가, 작품·세계관 관리, Fork 및 발행 처리
Content Service	Spring Boot, Kotlin	작품·세계관·회차·원고 관리, 버전·체크포인트·Fork·발행 처리, 독서·검색·작품 피드백 및 첨부 관리
Operations Service	Spring Boot, Kotlin	고객지원·공지·이벤트, 신고·소명·운영 판단·감사 기록 관리, 정기 작업 감시 및 LLMOps 관리
AI Service	FastAPI, Python, LangChain, LangGraph	세계관 설정 추출, AI 집필 지원, 다중 인물 시뮬레이션
RDB	PostgreSQL, Flyway,	회원·콘텐츠 등 업무 데이터와 원문·버전·변경 이력 저장
Graph DB	Neo4j	등장인물 및 세계관 엔티티 간 관계 저장·조회
파일 저장소	S3 호환 객체 스토리지	작품 표지·설정 이미지·원고 이미지 및 첨부 파일 저장
External AI	OpenAI API	LLM 추론, 대화 및 텍스트 생성
Infrastructure	Docker Compose, Nginx	서비스 컨테이너 관리 및 API 요청 프록시

5. 데이터 관리 원칙

STORYFORK는 서비스별 데이터 관리 책임을 구분하고, 원문·버전·관계 정보·파일의 특성에 맞는 저장소를 사용한다.

1. **서비스별 데이터 소유**:각 서비스는 담당 업무의 데이터를 관리하며, 다른 서비스의 데이터 변경은 해당 서비스의 API를 통해 요청한다.
2. **저장소 역할 구분**:PostgreSQL은 업무 데이터와 원문·버전·변경 이력을, 파일 저장소는 이미지와 첨부 파일을 보관한다. 관계 그래프 DB는 인물·세계관 관계의 저장·조회에 활용하도록 설계하며, 제품과 연동 방식은 별도로 결정한다.
3. **버전 및 분기 보존**:원고·설정의 변경 이력과 Fork의 기준 회차·버전을 보존하고, 원작과 분기 작품의 변경을 구분한다.
4. **접근 범위 보호**:작품 소유권과 공개 범위에 따라 데이터 접근을 제한하며, AI에는 해당 작업에 허용된 최소 문맥만 제공한다.

6. 설계 목표

STORYFORK는 다음 세 가지 목표를 중심으로 설계한다.

1. **서비스 책임 분리**:Core·Content·Operations·AI의 업무 경계를 명확히 하여 기능 변경의 영향을 줄이고 독립적인 개발과 유지보수를 지원한다.
2. **데이터 일관성과 이력 보존**:서비스별 정본 관리 책임을 명확히 하고, 원고·설정의 버전과 Fork 기준을 보존하여 변경 내용을 추적하고 복원할 수 있도록 한다.
3. **AI 실행과 정본 반영 분리**:AI의 생성 결과와 시뮬레이션 상태를 확정된 작품 데이터와 구분하고, 채택·저장·발행은 해당 업무의 승인 규칙에 따라 처리한다.

II. 시스템 구성 요소

1. 프론트엔드

프론트엔드는 사용자가 STORYFORK의 기능을 이용할 수 있도록 웹 기반 인터페이스를 제공한다.

항목	내용
기술 스택	React, TypeScript, Vite
아키텍처	Clean Architecture + MVVM
주요 기능	세계관 및 등장인물 관리, 작품 집필·발행, 작품 Fork, AI 시뮬레이션 결과 확인
통신 방식	REST API를 통해 Core·Content·Operations의 공개 API를 이용하며, AI 서비스와 데이터 저장소는 직접 호출하지 않는다.

2. 벡엔드 서비스

백엔드는 Core·Content·Operations·AI의 4개 서비스로 구성한다. 각 서비스는 담당 업무와 데이터 관리 책임을 분리하며, API를 통해 필요한 기능을 연동한다.

구분	Core API	Content Service	Operations Service	AI Service
언어	Kotlin	Kotlin	Kotlin	Python
프레임워크	Spring Boot	Spring Boot	Spring Boot	FastAPI
아키텍처	Layered Architecture	Layered Architecture	Layered Architecture	Layered + Agentic Workflow
주요 역활	회원·인증·세션, 크레딧·정산, 회원 알림	작품·세계관·원고·버전·Fork·발행, 독서·검색·작품 피드백·첨부	고객지원·공지·이벤트, 신고·소명·운영 판단·감사, 작업 감시·LLMOps	설정 추출·집필 지원·다중 인물 시뮬레이션·피드백 분석 등 AI 실행
데이터 관리	회원·인증·크레딧·알림 데이터	콘텐츠·버전·발행 상태·피드백·파일 참조	문의·신고 사건·운영 결정·감사 기록	AI 작업에 필요한 실행 상태와 결과

Core는 공통 회원 업무를, Content는 콘텐츠의 접근 권한과 생성·수정·발행을 담당한다. Operations는 운영 판단과 조치 요청을 관리하며, 실제 회원·콘텐츠 변경은 해당 데이터를 소유한 서비스가 처리한다. AI는 허용된 문맥에서 결과를 생성하고, 정본 수정·발행·정산의 최종 판단은 수행하지 않는다.

3. 데이터 저장소

STORYFORK는 관계형 데이터베이스와 그래프 데이터베이스를 조합하는 구조를 검토한다.

데이터베이스	저장 대상	주요 목적
PostgreSQL	사용자, 작품, 회차, 세계관 문서, 변경 이력, AI 작업 기록	정형 데이터 및 정본 관리
Neo4j	등장인물 관계, 인물 간 감정, 세계관 내 엔티티 연결	관계 그래프 탐색 및 분석
S3 호환 객체 스토리지	작품 표지·설정 이미지·원고 이미지 및 첨부 파일	파일 원본 보관 및 접근 제어된 제공

4. AI 모델 및 외부 연동

구분	적용 대상	주요 역할
외부 AI 모델	외부 모델 API — 현재 생성 경로는 OpenAI API 사용	텍스트 생성·추론 등 요청된 모델 기능 수행
AI 실행 구조	Python·FastAPI 기반, 에이전트 협업 구조 설계	작업별 AI 역할과 실행 상태 관리, 생성·분석 작업 수행
창작 지원	설정 추출, 집필 제안, 다중 인물 시뮬레이션	원고의 설정 정리, 장면·대화 생성 및 이야기 전개 탐색 지원
피드백 지원	공개 근거 기반 질문 응대·오류 수정안 제시, 작가 검토 지원	독자가 선택한 피드백 유형에 따라 답변·수정안·검토 자료 생성
데이터 연계	업무 서비스가 제공하거나 조회를 허용한 문맥 활용	사용자 권한·회차 공개 범위·인물별 지식 범위에 맞는 정보 사용

5. 인프라 및 실행 환경

각 서비스는 Docker 컨테이너로 독립 실행하며, Docker Compose를 이용해 개발 및 배포 환경을 구성한다.

구성 요소	적용 기술	역할
컨테이너	Docker	서비스별 실행 환경 분리
컨테이너 관리	Docker Compose	다중 서비스 실행 및 네트워크 관리
웹 서버	Nginx	정적 파일 제공 및 API 요청 프록시
서비스 통신	HTTP / REST API	Core API와 AI Service 간 내부 통신
데이터 관리	PostgreSQL / Neo4j	업무 데이터 및 관계 그래프 저장